

医疗大模型新人快速上手指南

作者：张沐寒 适用对象：刚进组、零编程基础的医学背景学弟学妹

💡 写在前面的话

你好，欢迎来到课题组。

我知道你现在可能正盯着一堆 `.py` 文件发呆，或者刚刚被一行红色报错吓到关掉了终端。没关系——我也经历过。

这份指南是我在完成两个课题挑战（**中文病例实体抽取** 和 **心衰患者生存预测与英文论文撰写**）后，把踩过的坑、走过的弯路，以及那些“早知道有人告诉我就好了”的经验，整理成的一份实战手册。

一、工具篇：搭好你的"手术台"

外科医生上台前要铺好无菌巾、摆好器械。做 AI 实验也一样——**环境没配好，后面的一切都是空谈。**

1.1 Python 环境配置：从零开始不恐慌

这一节你只需要安装两个软件：**Anaconda**（管理 Python 环境）和 **VS Code**（写代码的编辑器），跟着步骤一步一步来。

第一部分：安装 Anaconda

1. 打开浏览器，访问 Anaconda 官网：<https://www.anaconda.com/>

⚠️ 如果网页打不开或下载很慢，请先搭建梯子（方法后续讲解）后再操作。

2. 点击页面上的下载按钮，选择适合你系统的版本（Windows 选 `.exe`，Mac 选 `.pkg`）。

3. 下载完成后，双击安装包开始安装。安装过程按默认推荐选项点击"Next"即可，**但有一点必须注意：**

⚠️ **安装路径必须全为英文字母！** 不能有中文、空格或特殊符号。例如 `D:\anaconda3` 是安全的，`D:\我的软件\anaconda` 会导致后续报错。

4. 安装过程需要等待较长时间（5~15 分钟），耐心等待进度条走完即可。

5. 安装完成后，在 Windows 开始菜单中搜索 **"Anaconda Prompt"**，能打开一个黑色命令窗口说明安装成功。

📺 **参考视频：**如果以上步骤有不理解的地方，可参考 B 站教程（搜索"Anaconda安装教程 2024"），看到安装软件部分即可，**C++ 相关步骤无需操作。**

第二部分：安装并配置 VS Code

VS Code 是目前最流行的代码编辑器，界面友好，新手友好。

1. 访问 VS Code 官网下载安装包：<https://code.visualstudio.com/>，按默认选项安装完成。

2. **汉化界面**（强烈推荐）：

1. 打开 VS Code，点击左侧边栏第四个图标（四个小方块，即"扩展"按钮）。

2. 在搜索框中输入 `chinese`。
3. 找到 "Chinese (Simplified)" 插件，点击 **Install (安装)**。
4. 安装完成后重启，界面变为中文即成功。

3. 安装 Python 插件：

1. 同样在扩展搜索框中输入 `python`。
2. 安装微软官方出品的 **Python** 插件（图标为蓝色蟒蛇）。

第三部分：让 VS Code 找到 Anaconda 的 Python

💡 为什么要特意指定 Anaconda 里的 `python.exe`？

你的电脑上可能已经有一个"系统自带的 Python"，但它和 Anaconda 的环境是完全割裂的——你在 Anaconda 里安装的所有科研包（`pandas`、`transformers` 等），系统 Python 根本找不到。**不指定正确的解释器，代码永远都会报"找不到包"的错误。**

操作步骤如下：

1. 在 VS Code 中按下快捷键 `Ctrl + Shift + P`，顶部会出现一个命令搜索框。
2. 输入 `Python: Select Interpreter`（输入前几个字母会自动补全），按回车。
3. 在弹出的列表中，找到路径中包含 `anaconda` 字样的那一个，例如：

```
Python 3.x.x ('base': conda) ~/anaconda3/bin/python
```

点击选择它。

⚠️ 如果列表中没有出现 Anaconda 的选项，点击 "**Enter interpreter path...**"，手动粘贴 Anaconda 的 `python.exe` 路径。Windows 用户的路径通常类似于 `C:\Users\你的用户名\anaconda3\python.exe`。

第四部分：运行你的第一行代码

环境配置完成后让我们来验证一下。

1. 在 VS Code 中新建一个文件，命名为 `test.py`（注意后缀是 `.py`）。
2. 在文件中输入以下代码：

python

```
print("hello world")
```

1. 点击编辑器右上角的  **(运行)** 按钮，或在文件内右键选择 "**运行 Python 文件**"。
2. 在下方的终端面板中，看到输出即意味着配置完成：

```
hello world
```

1.2 国内无痛调用大模型 API

很多新人第一次尝试调用 GPT 的 API，结果发现需要境外信用卡、还要搭梯子——直接劝退。本节推荐一个国内直连、新用户有免费额度的替代方案：阿里云百炼平台的通义千问（Qwen）模型。

为什么推荐通义千问（Qwen）？

考量维度	说明
网络环境	国内直连，无需梯子，稳定可靠
免费额度	新用户注册后可领取免费 Token 额度，足够前期实验使用
中文能力	针对中文语料深度训练，对病历、医学术语的理解准确率高
调用方式	兼容 OpenAI 接口标准，学会后可无缝切换其他模型

第一步：注册并获取 API Key

1. 打开浏览器，访问阿里云百炼平台：<https://bailian.console.aliyun.com/>
2. 使用支付宝或淘宝账号登录（无需单独注册）。
3. 登录后，点击右上角头像，进入"API-KEY 管理"，点击"创建新的 API-KEY"。
4. 复制生成的 API Key，务必妥善保存（只显示一次！）。

第二步：安装所需的 Python 库

在 Anaconda Prompt（或 VS Code 的终端）中，激活你的环境后运行：

bash

```
# 安装 openai 库（阿里云百炼兼容 OpenAI 接口）
pip install openai
```

💡 调用通义千问用的是 openai 这个库。这意味着你学会这套写法，将来切换到其他模型也几乎不需要改动代码。

第三步：写你的第一个 API 调用代码

新建一个文件，命名为 qwen_test.py，将以下代码完整复制进去：

python

```
from openai import OpenAI

# =====
# 【通行证】API Key: 进入实验室的身份凭证
# 就像你的工牌——证明你是有权限调用模型的用户
# 请将引号内替换为你自己的 API Key
# =====
```

```

client = OpenAI(
    api_key="sk-xxxxxxxxxxxxxxxxxxxxxxxx", # 替换为你的 API Key
    base_url="https://dashscope.aliyuncs.com/compatible-mode/v1" # 阿里云百炼的接口地址
)

# =====
# 【会诊申请】构造发送给模型的消息
# messages 就像一份会诊单，告诉模型它的角色和你的问题
# =====
response = client.chat.completions.create(
    # 【科室选择】model: 决定调用哪个"专科医生"（模型版本）
    # qwen-plus 性价比高，适合日常任务；qwen-max 能力更强但消耗 Token 更多
    model="qwen-plus",

    messages=[
        {
            # system: 给模型设定角色，相当于告诉会诊医生"你是心内科专家"
            "role": "system",
            "content": "你是一位专业的医疗信息提取助手，擅长从中文病历中提取结构化信息。"
        },
        {
            # user: 你真正想问的问题，相当于会诊单上的"会诊目的"
            "role": "user",
            "content": (
                "请从以下病历片段中，提取\"主诉\"、\"诊断\"两项信息，"
                "以 JSON 格式输出。\\n\\n"
                "【病历】患者，男，72岁，因'反复胸闷、气促5年，加重伴双下肢水肿1周'入院。"
                "既往高血压病史20年。入院诊断：慢性心力衰竭急性加重期。"
            )
        }
    ]
)

# =====
# 【读取报告】提取模型返回的文字内容
# =====
result = response.choices[0].message.content
print(result)

```

运行后，你应该会看到类似下面的输出：

json

```

{
  "主诉": "反复胸闷、气促5年，加重伴双下肢水肿1周",
  "诊断": "慢性心力衰竭急性加重期"
}

```

至此你已经完成了第一次大模型 API 调用。

⚠️ 避坑 Tips: API Key 安全须知

请务必遵守以下原则：

1. **绝对不要上传到 GitHub 等公开仓库。** GitHub 会自动扫描并警告 API Key 泄露。一旦泄露，他人可以用你的额度无限调用，产生的费用由你承担。
2. **使用环境变量存储，而非硬编码在代码里。** 正确做法：

python

```
import os

# 从系统环境变量中读取 API Key，而非直接写在代码里
api_key = os.environ.get("DASHSCOPE_API_KEY")
client = OpenAI(api_key=api_key, base_url="https://dashscope.aliyuncs.com/compatible-mode/v1")
```

在终端中设置环境变量（每次开新终端需重新设置，或写入系统配置文件）：

bash

```
# Windows (Anaconda Prompt)
set DASHSCOPE_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxx

# Mac / Linux
export DASHSCOPE_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

Token 用量小贴士

调用 API 是按 **Token**（可以粗略理解为"字数"）计费的。在免费额度期间，注意以下几点可以节省用量：

- `system` 提示词尽量简洁，不要长篇大论
- 不需要历史对话时，每次调用都重新构建 `messages`，不要无限累积上下文
- 上传文件时先压缩删除无效间隔与空白段落等

1.3 AI 工具选择与海外服务配置

很多新人卡在这里：ChatGPT 打不开，Claude 注册不了，Gemini 一直转圈。这不是你的错，是网络问题。本节先解决"能不能上"，再解决"怎么用好"。

第一部分：先搞清楚你需要什么

访问 ChatGPT、Claude、Gemini 这类海外 AI 服务，需要两样东西：

一个"梯子"客户端软件（如 Clash Verge Rev）
+
一份"机场"节点订阅（提供真实的海外服务器）

💡 **类比解释：**客户端软件就像医院里的**叫号系统**，本身不能看病；机场节点才是真正的**医生**（海外服务器）。两者缺一不可。

第二部分：搭建网络环境（Clash Verge Rev）

步骤一：下载并安装 Clash Verge Rev

- 访问官网：<https://clash-verge-rev.org/install/install.html>
- 根据你的系统选择版本，下载后双击安装，首次启动会请求管理员权限，**点击允许**。

步骤二：购买机场服务并导入订阅

机场是提供海外节点的订阅服务，按月或按流量计费。以下是团队亲测可用的选项：

机场名称	价格参考	访问门槛	特点
Azabudai	见官网	● 无需梯子可直接访问	适合新人首选，注册方便
SakuraCat	见官网	● 需梯子（暨大校园网可直接访问）	稳定性较好
海豚云	¥90/季（约750G）	需梯子	量大，支持6台设备，可签到领流量

建议：**先从** Azabudai**（无需梯子即可注册）入手，购买后复制页面上的“订阅链接”备用。

导入订阅步骤：

- 打开 Clash Verge Rev，点击左侧菜单**“订阅”**。
- 点击**“新建”**，将机场后台复制的订阅链接粘贴进去。
- 点击保存，等待节点列表加载完成。
- 选择一个延迟较低（绿色）的节点，点击激活。

步骤三：配置代理模式（重要！）

Clash 有三种工作模式，日常科研推荐如下设置：

模式	什么时候用	说明
规则模式 ☒ 日常推荐	正常学习、写代码	自动判断：国内网站直连，境外网站走代理，两全其美
全局模式	临时排错	所有流量走代理，国内网站变慢， 不要用来观看视频
直连模式	关闭代理时	等同于完全不使用梯子

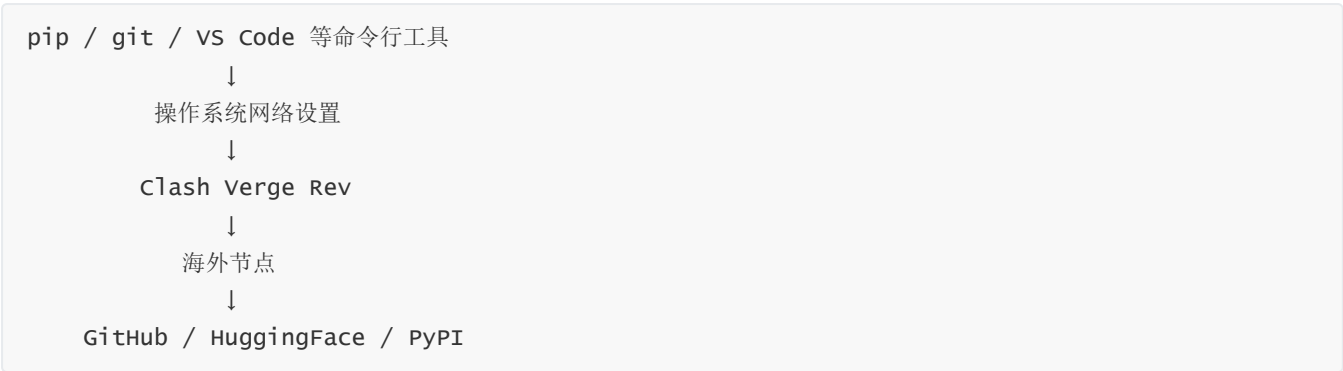
警告：禁止用全局模式观看视频（B 站、爱奇艺等），会极速消耗机场流量，套餐分分钟用光。

步骤四：开启系统代理（关键！）

不开系统代理，浏览器可能能上网，但 `pip install`、**VS Code 插件下载** 等命令行操作仍然会超时失败。

在 Clash Verge Rev 中，找到**"系统代理"**开关，确认它处于**开启状态**。

开启后，网络请求的路径变为：



💡 何时需要 TUN 模式：如果开了系统代理，某些工具（如 Docker）仍然无法联网，可以在 Clash 设置中额外开启 **TUN 模式**。它通过创建虚拟网卡在网络底层拦截所有流量，兼容性更强，但对电脑性能要求稍高，日常不强制开启。

步骤五：检测网络质量（可选但推荐）

搭好梯子后，建议访问 <https://ippure.com> 检测当前节点的 IP 质量。

指标	含义	对科研的影响
IPPure 系数	该 IP 的全网黑历史评分，越低越好	数值过高会导致注册 ChatGPT/Claude 账号时被封
CF 系数	Cloudflare 风险评分，越低越好	数值过高会频繁触发"我不是机器人"验证码，影响效率

💡 如果注册账号屡屡失败，不是你操作有问题，很可能是当前节点 IP 太"脏"——换一个节点再试。

第三部分：三大 AI 工具的正确打开方式

网络搭好之后，来认识这三位常用ai工具。

工具定位总览

工具	访问地址	国内直连	免费版	最擅长的事
ChatGPT	chat.openai.com	❌ 需梯子	有限制	快速代码生成、数学推导、广泛知识问答
Claude	claude.ai	❌ 需梯子	有限制	逻辑严密的代码梳理、论文润色
Gemini	gemini.google.com	❌ 需梯子	有限制	结合 Google 搜索的实时信息查询、多模态任务

提问模板：让 AI 真正帮到你

AI 不是搜索引擎，**提问的质量决定回答的质量**。用下面这个prompt模板，把你的问题说清楚：

【我的背景】我是医学背景的科研新人，不熟悉编程。

【我的目标】我想实现：[用一句话描述]

【我的环境】Python [版本]，已安装的包：[列出来]

【当前问题】[粘贴代码 / 粘贴报错信息 / 描述现象]

【我希望的输出格式】

- 可以直接运行的完整代码
- 每个关键步骤附一行中文注释
- 指出我原来代码哪里错了

第四部分：使用海外 AI 服务的安全须知

**** 切换节点前，务必先关闭所有正在运行的 ChatGPT、Claude、Gemini 等网页。 **** 在登录状态下切换 IP，平台的风控系统可能将你的账号判定为异常登录，轻则要求重新验证，重则直接封号。

养成以下习惯，保护你的账号：

1. **固定使用同一个或同一地区的节点登录同一个账号**，减少 IP 跳变触发风控。
2. **不要在他人电脑上登录自己的账号**，用完不退出是封号高危行为。
3. **免费额度用完后不要频繁注销重注**，平台有检测机制。

二、实战篇：让 AI 成为你的编程搭档

报错不是失败的信号，真正的差距不在于你会不会写代码，而在于你会不会向 AI 提问。本章教你掌握与 AI 协作 Debug 的方法论。

2.1 Debug 方法论：为什么你的提问没有得到好答案

很多人遇到报错后，第一反应是把错误信息单独截图发给 AI，然后问："为什么它给不出有用的答案？"

原因很简单：你给 AI 的信息，远不足以让它做出准确判断。

💡 **医学类比：**想象一位急诊科医生只收到一句话——"病人很不舒服"——他能开出正确的处方吗？他需要的是：主诉、现病史、体征、检查结果.....AI 也一样，它需要的是完整的"病历"，而不是一句"我的代码报错了"。

AI Debug 需要你提供

一：代码上下文（不是只发报错那一行）

报错的那行代码往往只是"症状"，真正的"病因"可能在十行之前的变量定义里。**必须提供完整的、可复现问题的代码片段**，而非单行截图。

二：完整的错误追溯 (Traceback)

Traceback 是系统给出的错误调用链，每一行都是线索。很多人只复制最后一行红色文字，丢失了最关键的文件路径和行号信息。**请完整复制从** `Traceback (most recent call last):` **开始的全部内容。**

三：数据样本（哪怕只有几行）

尤其是处理临床数据集时，同样的代码在不同数据上可能有完全不同的表现。**提供 3~5 行真实（或脱敏后的）数据样本**，让 AI 能理解你的数据结构，而不是凭空猜测。

2.2 三个核心技巧：

技巧一：给 AI 设定专业身份（角色设定）

在对话开始时，用一句话告诉 AI 它是谁、你是谁、任务是什么。

没有角色设定的提问，AI 会给出泛化的通用答案；有了角色设定，它会用更专业、更贴合你场景的语言来回应。

- 不带角色设定（泛化）：“帮我修这段 Python 代码”
- 带角色设定（精准）：“你是一位精通 Python 数据科学的医疗 AI 研究员，熟悉 pandas、scikit-learn 和临床数据集的特点。我是医学背景的科研新人，正在处理心衰患者的生存预测任务。请帮我修复以下代码……”

技巧二：在当前对话中迭代纠偏（不要动不动重开）

AI 的上下文（Context）是它最宝贵的资源。在同一个对话中，AI 记得你之前分享的代码、报错和所有背景信息。每次重开，它就“失忆”了，你要把所有背景重新铺垫一遍。

正确的迭代姿势是：在同一对话里，用追问来精准纠偏。

场景	❌ 低效做法	✅ 高效做法
代码还是报同样的错	关窗口，重新问	"按你的方案修改后，仍然报同样的错：[粘贴新报错]，请继续排查"
解释看不懂	换个 AI 问	"你上面提到的 dtype 转换我不理解，能用心衰数据集的例子重新解释一遍吗？"
代码能跑但结果不对	放弃，重开	"代码运行成功，但输出结果是 [X]，我预期应该是 [Y]，可能哪里逻辑有误？"
想要更详细的解释	另开新对话	"请逐行解释你刚才修改的代码，重点说明每处改动的原因"

技巧三：先梳理思路，再写代码

在面对一个全新任务时，**不要让 AI 直接帮你生成代码**。先让它帮你把处理逻辑梳理清楚，你理解了之后再动手。在开始搭框架之前，甚至可以让 AI 反问你一些问题，以确保它充分理解你的需求。

- 梳理思路型 Prompt 示例：
- "我有一份心衰患者临床数据集（CSV 格式），包含年龄、血清肌酐、射血分数、随访时间、是否死亡等字段。我想用随机森林模型预测患者生存结局。在你帮我写代码之前，请先帮我梳理：

1. 数据预处理应该按什么顺序做？
2. 哪些字段可能需要特殊处理（如类型转换、缺失值填充）？
3. 特征工程有哪些常见策略适合这类数据？请用简洁的步骤列表回答，暂时不需要写代码。"

拿到逻辑框架，确认你理解并认可之后，再说："好，现在按这个框架帮我写完整的 Python 代码。"

2.3 万能 Prompt 模板

将以下五个模块填写完整，复制粘贴给 AI，90% 的编程问题都能在一轮对话内解决。

【身份设置】 你是一位精通 Python 数据科学的医疗 AI 研究员，熟悉 pandas、scikit-learn、XGBoost，以及临床数据集的常见问题（缺失值、数据类型不一致、列名错误等）。

【任务背景】 我是医学背景的科研新人，正在完成一个使用心衰患者临床数据训练随机森林分类模型、预测患者生存结局的任务。

【代码片段】

python

粘贴你的完整代码（不要只粘贴报错那一行）

【报错内容】

粘贴完整的 Traceback，从第一行到最后一行，一字不漏

【数据样本】

粘贴 df.head() 或 print(df.dtypes) 的输出结果
如涉及真实患者数据，请先脱敏（删除姓名、住院号等）

【预期结果】 我希望代码能够。

【输出要求】 请：① 指出报错的根本原因；② 给出修复后的完整代码；③ 在修改处用注释标注"改动原因"；④ 如果还有潜在风险，请一并指出。

2.4 真实案例复盘：一次心衰数据建模的报错全程

背景

在挑战二（心衰患者生存预测）中，我拿到一份心衰临床数据集，尝试用随机森林训练分类模型，结果在特征工程阶段遭遇了连续两个报错。以下是完整的问题复盘，以及"低质量 vs 高质量"提问方式的真实对比。

报错一：列名 KeyError

问题场景：读取数据后，尝试选取特征列时报错。

python

```
import pandas as pd
from sklearn.ensemble import RandomForestClassifier

df = pd.read_csv("heart_failure_clinical_records.csv")

# 尝试选取特征
features = df[["age", "anaemia", "creatinine_phosphokinase",
               "diabetes", "ejection_fraction", "high_blood_pressure",
               "platelets", "serum_creatinine", "serum_sodium",
               "sex", "smoking", "time", "DEATH_EVENT"]]
```

报错信息：

```
KeyError: "None of [Index(['DEATH_EVENT'], dtype='object')] are in the [columns]"
```

✖ **低质量提问：**

"我的代码报了 KeyError，请帮我修复"

✔ **高质量提问：**

你是一位精通 pandas 和临床数据处理的数据科学家。我是医学背景的新人，正在处理心衰数据集。

【代码片段】

python

```
df = pd.read_csv("heart_failure_clinical_records.csv")
features = df[["age", "anaemia", "creatinine_phosphokinase",
               "diabetes", "ejection_fraction", "high_blood_pressure",
               "platelets", "serum_creatinine", "serum_sodium",
               "sex", "smoking", "time", "DEATH_EVENT"]]
```

【报错内容】

```
KeyError: "None of [Index(['DEATH_EVENT'], dtype='object')] are in the [columns]"
```

【数据样本】

```
print(df.columns.tolist()) 的输出：
['age', 'anaemia', 'creatinine_phosphokinase', 'diabetes',
 'ejection_fraction', 'high_blood_pressure', 'platelets',
 'serum_creatinine', 'serum_sodium', 'sex', 'smoking', 'time', 'DEATH_EVENT ']
```

请帮我找出 KeyError 的根本原因并修复。

AI 有了数据样本，立刻就能发现：`df.columns.tolist()` 输出里，`'DEATH_EVENT '` 末尾有一个隐藏空格

AI 给出的修复代码：

python

```
# 第一步：清理所有列名的首尾空格（处理 CSV 导入的隐藏脏数据）
df.columns = df.columns.str.strip() # ← 改动原因：列名含隐藏空格导致 KeyError

# 第二步：再进行列选取，此时列名已干净
x = df.drop(columns=["DEATH_EVENT"]) # 特征矩阵
y = df["DEATH_EVENT"]               # 标签列
```

报错二：数据类型不一致 (String vs Float)

紧接着，在训练模型时又遇到了新问题：

python

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X, y)
```

报错信息：

```
ValueError: could not convert string to float: 'Yes'
```

✅ 在同一对话中继续追问（迭代纠偏）：

"按你的方案清理列名后，KeyError 已解决，但训练时又报了新错误：

```
ValueError: could not convert string to float: 'Yes'
```

`X.dtypes` 的输出如下：

age	float64	
anaemia	object	← 应该是 0/1，但变成了 Yes/No
diabetes	object	← 同上
high_blood_pressure	object	← 同上
ejection_fraction	float64	
...		

请继续帮我排查并修复。"

AI 立刻定位：部分本应为 0/1 的二分类字段，因数据录入时使用了 `Yes/No` 字符串，导致 scikit-learn 无法将其转换为浮点数进行计算。

AI 给出的修复代码：

python

```
# 找出所有 object 类型的列（即字符串列）
string_cols = X.select_dtypes(include="object").columns
print("需要处理的字符串列：", string_cols.tolist())

# 将 Yes/No 映射为 1/0（改动原因：sklearn 不接受字符串特征）
yes_no_map = {"Yes": 1, "No": 0}
X[string_cols] = X[string_cols].replace(yes_no_map)

# 验证：确认没有剩余的字符串列
assert X.select_dtypes(include="object").empty, "仍有未处理的字符串列！"

# 现在可以安全训练
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X, y)
print("模型训练完成！")
```

2.5 进阶技巧：让 AI 不只修复，还要教会你

技巧：要求 AI 逐行解释改动原因

拿到修复后的代码，不要只运行了事。让 AI 逐行解释为什么这么改，才是真正的学习。

追问模板：

"代码已经跑通了，谢谢！但我还不完全理解以下几行的含义，请逐行解释，并告诉我如果不这样写会出现什么问题：

python

```
df.columns = df.columns.str.strip()
X[string_cols] = X[string_cols].replace(yes_no_map)
assert X.select_dtypes(include='object').empty
```

请用适合医学背景读者的语言解释，可以用临床场景做类比。"

这个习惯养成后，你会发现每解决一个报错，你就真正学会了一个数据处理的知识

常见报错类型速查表

报错类型	中文含义	医疗数据集中的典型场景
KeyError	找不到该列名	列名含隐藏空格、大小写不匹配 (DEATH_EVENT vs death_event)
ValueError: could not convert string to float	字符串无法转为数字	Yes/No、男/女等文字字段未编码
UnicodeDecodeError	编码格式错误	医院导出的 GBK 编码 CSV 用 UTF-8 读取

报错类型	中文含义	医疗数据集中的典型场景
<code>json.JSONDecodeError</code>	JSON 解析失败	大模型返回了带 Markdown 代码块的字符串
<code>ValueError: Input contains NaN</code>	数据含空值	临床数据中的缺失检验结果未填充
<code>MemoryError</code>	内存不足	数据集过大，或模型参数设置不当

2.6 实战案例补充：病历实体抽取中的 JSON 解析问题

背景：在挑战一（中文病历实体抽取）中，调用大模型 API 并要求返回 JSON 格式结果，但模型经常在 JSON 外面包裹 Markdown 代码块，导致直接解析失败。

报错现象：

python

```
import json

response_text = response.choices[0].message.content
# 模型实际返回的内容长这样：
# ```json
# {"疾病": ["高血压", "心力衰竭"], "症状": ["心悸", "气促"]}
# ```

result = json.loads(response_text) # 报错！
json.JSONDecodeError: Expecting value: line 1 column 1 (char 0)
```

根本原因：模型"礼貌性"地用 Markdown 代码块包裹了 JSON，但 `json.loads()` 只能解析纯 JSON 字符串，无法处理 ````json` 这样的格式符号。

修复方案：

python

```
import re
import json

def safe_parse_json(text: str) -> dict:
    """
    安全解析大模型返回的 JSON 字符串。

    改动原因：
    - re.sub 用于剥离模型可能添加的 Markdown 代码块标记
    - try/except 确保解析失败时不会让整个程序崩溃
    - 返回空字典而非抛出异常，便于后续逻辑继续处理
    """
    # 去除 ```json ... ``` 或 ``` ... ``` 包裹
    text = re.sub(r"```(?:json)?\s*", "", text).strip()
    text = text.replace("```", "").strip()
```

```
try:
    return json.loads(text)
except json.JSONDecodeError as e:
    print(f"⚠️ JSON 解析失败, 原始内容: \n{text}\n错误信息: {e}")
    return {}
```

使用示例

```
result = safe_parse_json(response_text)
```

```
print(result)
```

```
# 输出: {"疾病": ["高血压", "心力衰竭"], "症状": ["心悸", "气促"]}
```

三、进阶篇：让 AI 成为你的编程搭档

本章教你用 AI 生成期刊级图表、解决"图表文字变乱码"的幻觉问题，以及用 LaTeX 写出标准的学术公式。

3.1 关于我用 AI 生成科研图表的实战经历

科研绘图的核心工作流只有四步：

① 告诉 AI 你的数据结构和图表目标（自然语言）

↓

② AI 生成可运行的 Python 代码

↓

③ 本地运行，根据效果在同一对话中迭代微调

↓

④ 导出高分辨率图→ 粘贴进论文

安装必要的库（一次性执行）：

bash

```
pip install matplotlib seaborn scikit-learn pandas numpy
```

图表一：ROC 曲线（标注 AUC 值）

ROC 曲线是评估二分类模型（如心衰死亡预测）最标准的图表，**必须标注 AUC 值**，否则审稿人会直接要求补充。

向 AI 提问的 Prompt：

你是一位精通医学数据可视化的数据科学家，熟悉医学期刊（如 JAMA、BMJ）的图表规范。

请帮我用 Python 绘制一张对比三个模型（随机森林、XGBoost、逻辑回归）的 ROC 曲线，要求：

- 图中标注每条曲线对应的 AUC 值，格式为 `Random Forest (AUC = 0.xx)`
- 添加对角参考线，使用灰色虚线
- 色盲友好配色（蓝、橙、绿），线宽 2pt
- 坐标轴标签：X 轴"False Positive Rate"，Y 轴"True Positive Rate"
- 图例放在右下角（`loc='lower right'`）

- 图片尺寸 6×5 英寸, DPI 300, 白色背景
- 请包含示例数据生成部分, 代码可以直接运行

AI 给出的参考代码:

python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_curve, auc

# — 生成心衰模拟数据 —————
X, y = make_classification(n_samples=300, n_features=12,
                           random_state=42)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

# — 训练三个模型 —————
models = {
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "XGBoost": GradientBoostingClassifier(random_state=42),
    "Logistic Reg.": LogisticRegression(max_iter=1000)
}
colors = ["#1f77b4", "#ff7f0e", "#2ca02c"] # 色盲友好配色

# — 绘图 —————
fig, ax = plt.subplots(figsize=(6, 5), dpi=300)

for (name, model), color in zip(models.items(), colors):
    model.fit(X_train, y_train)
    y_score = model.predict_proba(X_test)[:, 1]
    fpr, tpr, _ = roc_curve(y_test, y_score)
    roc_auc = auc(fpr, tpr)
    # 在图例中直接标注 AUC 值
    ax.plot(fpr, tpr, color=color, lw=2,
            label=f"{name} (AUC = {roc_auc:.3f})")

# 随机猜测基准线 (灰色虚线)
ax.plot([0, 1], [0, 1], color="gray", lw=1.5,
        linestyle="--", label="Random Guess")

ax.set_xlim([0.0, 1.0])
ax.set_ylim([0.0, 1.05])
ax.set_xlabel("False Positive Rate", fontsize=12)
ax.set_ylabel("True Positive Rate", fontsize=12)
ax.set_title("ROC Curves - Heart Failure Prediction", fontsize=13)
ax.legend(loc="lower right", fontsize=10)
ax.spines[["top", "right"]].set_visible(False) # 去掉多余边框
```



```
plt.tight_layout()
plt.savefig("roc_curve.png", dpi=300, bbox_inches="tight")
plt.show()
print("✅ 图片已保存为 roc_curve.png")
```

图表二：相关性热力图 (Correlation Heatmap)

热力图用于展示临床变量之间的相关性，在特征工程阶段帮助你识别冗余特征。

向 AI 提问的 Prompt:

请帮我用 seaborn 对心衰临床数据集 (heart_failure_clinical_records.csv) 绘制相关性热力图，要求：

- 使用 Pearson 相关系数，数值标注在每个格子中，保留两位小数
- 配色方案使用"冷暖双色" (coolwarm)，中心值 0 为白色
- 图表标题: "Feature Correlation Matrix – Heart Failure Dataset"
- 列名如果太长，自动旋转 45 度避免重叠
- 尺寸 10×8 英寸，DPI 300

参考代码:

python

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# 读取数据 (确保先处理好列名空格)
df = pd.read_csv("heart_failure_clinical_records.csv")
df.columns = df.columns.str.strip() # 清除列名隐藏空格

# 计算 Pearson 相关矩阵
corr_matrix = df.corr(method="pearson")

# 绘制热力图
fig, ax = plt.subplots(figsize=(10, 8), dpi=300)

sns.heatmap(
    corr_matrix,
    annot=True,           # 在每格显示数值
    fmt=".2f",            # 保留两位小数
    cmap="coolwarm",     # 冷暖双色, 0 值为白色
    center=0,            # 相关系数 0 对应白色
    linewidths=0.5,      # 格线宽度
    linecolor="white",    # 正方形格子
    square=True,
    ax=ax
)

ax.set_title("Feature Correlation Matrix – Heart Failure Dataset",
             fontsize=13, pad=15)
```

```
plt.xticks(rotation=45, ha="right", fontsize=9)
plt.yticks(rotation=0, fontsize=9)

plt.tight_layout()
plt.savefig("correlation_heatmap.png", dpi=300, bbox_inches="tight")
plt.show()
print("✅ 热力图已保存")
```

图表三：特征重要性条形图

用随机森林或 XGBoost 训练完模型后，特征重要性图能直观展示“哪些临床指标对预测最关键”，是论文结果部分的重要图表。

向 AI 提问的 Prompt:

请帮我用 matplotlib 绘制随机森林模型的特征重要性水平条形图，要求：

- 按重要性从高到低排序，最重要的特征在最顶部
- 条形颜色根据重要性深浅渐变（越重要越深）
- X 轴标签为"Feature Importance (Gini Impurity)"
- 每条柱右侧标注具体数值（保留三位小数）
- 尺寸 8×6 英寸，DPI 300，白色背景

参考代码：

python

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.cm as cm
from sklearn.ensemble import RandomForestClassifier

# （假设已完成数据准备，X 和 y 已定义）
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# 提取特征重要性并排序
feature_names = X_train.columns.tolist()
importances = model.feature_importances_
indices = np.argsort(importances) # 从低到高排序（绘图时顶部最高）

# 颜色渐变：重要性越高，颜色越深
colors = cm.Blues(np.linspace(0.3, 0.9, len(importances)))
sorted_colors = colors[np.argsort(np.argsort(importances))]

fig, ax = plt.subplots(figsize=(8, 6), dpi=300)

bars = ax.barh(
    range(len(indices)),
    importances[indices],
```

```
        color=sorted_colors,
        edgecolor="white"
    )

    # 在每条柱右侧标注数值
    for i, (bar, idx) in enumerate(zip(bars, indices)):
        ax.text(bar.get_width() + 0.002, bar.get_y() + bar.get_height() / 2,
                f"{importances[idx]:.3f}",
                va="center", ha="left", fontsize=9)

    ax.set_yticks(range(len(indices)))
    ax.set_yticklabels([feature_names[i] for i in indices], fontsize=10)
    ax.set_xlabel("Feature Importance (Gini Impurity)", fontsize=11)
    ax.set_title("Random Forest - Feature Importance\n(Heart Failure Dataset)",
                fontsize=12)
    ax.spines[["top", "right"]].set_visible(False)

plt.tight_layout()
plt.savefig("feature_importance.png", dpi=300, bbox_inches="tight")
plt.show()
print("✅ 特征重要性图已保存")
```

3.2 科研 AI 绘图的通用方法论

第一关：先搞清楚"这张图该用什么方式画"

科研绘图有两条完全不同的技术路线，选错了会让你在错误的方向上浪费大量时间：

路线	代表工具	核心原则	典型图表
代码生成（数据驱动）	Python + matplotlib/seaborn	数值必须来自你的真实数据，AI 只负责写代码	ROC 曲线、热力图、散点图、生存曲线、混淆矩阵
AI 直接生成（概念驱动）	Gemini	传达逻辑和视觉风格，不依赖精确数值	研究框架图、诊断流程图、解剖示意图、方法论图

判断方法很简单——如果这张图里的数字有我从数据集里算出来的，就必须用代码生成。

第二关：写出让 AI 生成高质量绘图代码的 Prompt

同样是让 AI 写绘图代码，Prompt 质量的差距，可能决定你拿到"能跑但丑"的代码，还是"开箱即用的期刊级图表"。

绘图 Prompt 的五要素：

- ① 我的数据是什么形态（别让 AI 猜）
- ② 我要画什么类型的图（越具体越好）
- ③ 视觉规范要求（颜色、字体、尺寸、DPI）
- ④ 内容标注要求（数值标注、图例位置、参考线）
- ⑤ 输出要求（包含示例数据、代码可直接运行）

❌ 低质量 Prompt：

"帮我画一个 ROC 曲线"

✅ 高质量 Prompt:

你是一位精通医学数据可视化的数据科学家，熟悉 Nature/BMJ 期刊的图表规范。

① **数据形态**：我有一个二分类预测任务，已经用 scikit-learn 训练了三个模型，每个模型都能输出预测概率（`predict_proba`）和真实标签。

② **图表类型**：三条 ROC 曲线对比图，每条对应一个模型。

③ **视觉规范**：白色背景、无顶部和右侧边框（`spines` 隐藏）、色盲友好配色（蓝 `#1f77b4`、橙 `#ff7f0e`、绿 `#2ca02c`）、线宽 2pt、尺寸 6×5 英寸、DPI 300。

④ **内容标注**：图例中直接标注 AUC 值，格式为 `模型名 (AUC = 0.xxx)`；添加灰色虚线对角参考线；X/Y 轴标签用英文。

⑤ **输出要求**：请包含从零生成模拟数据的完整代码，确保直接运行即可看到图；在关键参数处添加中文注释。

第三关：拿到代码之后，不要直接运行——先做这三件事

拿到 AI 生成的代码，一键运行，报错了又茫然。养成以下三个习惯，可以节省大量时间：

① 先检查导入语句

python

```
# 看看 AI 用了哪些库，确认你已经安装
import matplotlib.pyplot as plt    # ← pip install matplotlib
import seaborn as sns              # ← pip install seaborn
from sklearn.metrics import roc_curve, auc # ← pip install scikit-learn
```

只需扫一遍 `import` 行，没安装的先 `pip install`，再运行。

② 把模拟数据替换成你的真实数据

AI 生成的代码通常包含用 `make_classification()` 或随机数组生成的模拟数据，这只是为了让代码可以直接运行。你需要找到这部分，替换成你自己的 `X_train`、`y_test`、`y_score` 等变量。

python

```
# AI 生成的模拟数据部分（找到它，替换掉）
X, y = make_classification(n_samples=300, ...) # ← 删掉这块
X_train, X_test, y_train, y_test = train_test_split(...) # ← 删掉这块

# 替换成你自己的数据
# y_test = df["DEATH_EVENT"]    (你的真实标签)
# y_score = model.predict_proba(X_test)[:, 1] (你的真实预测概率)
```

③ 先用小数据跑通，再用全量数据

第一次运行时，在数据加载后加一行：

python

```
df = df.sample(50, random_state=42) # 先只用 50 行测试，跑通了再去掉这行
```

这样即使有问题，报错也会在几秒内出现，不会等了 5 分钟才发现有 bug。

第四关：图跑出来了，但"不好看"——用 AI 迭代微调

第一版图表跑出来之后，通常需要至少 2~3 轮微调。**关键原则：在同一个对话里追问，不要重开。**

常见微调需求 & 追问模板：

问题	追问 Prompt
图例文字太小	"请将图例字体改为 11pt，并在图例外加一个细边框"
标题和图之间太近	"请在标题和图之间增加间距（pad=20）"
中文显示为方框	"中文显示乱码，请在代码开头添加设置中文字体的代码（系统是 Windows）"
想换配色方案	"请将配色改为 Nature 期刊风格：主色调为深蓝 #003f5c、橙红 #bc5090、金黄 #ffa600"

第五关：用 Gemini 生成概念性示意图（解决文字乱码问题）

当你需要的不是数据图表，而是**诊断流程图、方法论示意图**这类"概念性图片"时，推荐使用 Gemini) 直接生成，而非写代码。

通用 Prompt 框架（适合任何课题的框架图）：

请生成一张科研方法论框架图，要求：

内容结构：

分为三个横向阶段——[阶段一名称]、[阶段二名称]、[阶段三名称]，每个阶段下方列出 2~3 个子模块，用中文标注。

视觉风格：

扁平化设计，白色背景，主色调为蓝色系，模块间用带箭头的连接线表示流程走向。

字体：

标题加粗，子模块使用正常字重，所有中文清晰可读。

尺寸：

横向宽幅（16:9 比例），适合插入 PPT 或论文。

适合用 Gemini 生成 vs 必须用代码生成的图表对比：

图表类型	推荐方式	原因
研究框架图、技术路线图	Gemini 直接生成	传达逻辑关系，不依赖精确数值
诊断流程图、决策树示意图	Gemini 直接生成	需要文字标注 + 箭头逻辑，Gemini 渲染清晰
ROC 曲线、混淆矩阵	必须用代码	数值来自真实数据，AI 无法生成正确数字
热力图、特征重要性图	必须用代码	同上

不满意时，在同一对话里追问微调：

"图中左侧第一个模块的中文标注偏小，请放大字号并加粗"

"三个阶段之间的箭头改为双向箭头，表示双向影响关系"

"整体色调太深，请改用更浅的蓝色系，背景保持白色"

3.3 学术公式排版：让 AI 写出标准 LaTeX 代码

为什么不能直接复制 Word 里的公式？

医学 AI 论文提交给期刊（或用 Overleaf 写论文）时，**公式必须用 LaTeX 代码编写**，而不是 Word 里的"插入公式"对象。两者格式完全不兼容。

以 F1 Score 为例：

F1 Score 是评估实体抽取、分类模型时最重要的综合性指标（综合考量查准率和查全率），其计算公式为：

$$F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

向 AI 提问的 Prompt：

请帮我写出以下公式的标准 LaTeX 代码，要求：

1. F1 Score 的计算公式（含 Precision 和 Recall 的定义展开）
2. 公式编号为 (1)
3. 适合粘贴进 Overleaf（使用 `equation` 环境）
4. 同时给出可以在 Typora / Markdown 文件中预览的行内版本

AI 给出的 LaTeX 代码：

latex

```
% — 粘贴进 Overleaf 的完整公式（带编号） —————
\begin{equation}
  F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}
  \label{eq:f1_score}
\end{equation}

% 展开版（含 TP / FP / FN 定义）
\begin{equation}
  F_1 = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}
  \label{eq:f1_expanded}
\end{equation}

% 引用公式时使用（在正文中写）：
% As shown in Equation~\ref{eq:f1_score}, ...
% — 粘贴进 Typora / Markdown 的行内版本 —————
$$
F_1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}
$$
```

让 AI 帮你检查 LaTeX 语法

写好公式后，可以把代码发给 AI 进行检查：

"请检查以下 LaTeX 代码是否有语法错误，并告诉我在 Overleaf 编译时是否需要引入额外的宏包：

latex

[粘贴你的代码]
~~~"

常用需要在 Overleaf 引入的宏包（粘贴到导言区）：

latex

```
\usepackage{amsmath}    % 支持高级数学公式环境
\usepackage{amssymb}    % 支持额外数学符号
\usepackage{booktabs}   % 专业三线表格（投稿必备）
\usepackage{graphicx}   % 支持插入图片
```

## 3.4 AI 幻觉的识别与防范

在医疗科研场景中，幻觉不只是技术缺陷，可能是临床风险。 本节聚焦于科研过程中最常遇到的几类幻觉。

### 幻觉类型速查

| 幻觉类型 | 具体表现                 | 高危场景             | 防范方法                               |
|------|----------------------|------------------|------------------------------------|
| 文献幻觉 | 捏造论文标题、作者、期刊名        | 让 AI 推荐参考文献      | 每条文献必须在 PubMed/Google Scholar 逐一核实 |
| 数值幻觉 | 给出"看起来合理"但并非你真实数据的数字 | 让 AI 直接生成图表（非代码） | 所有图表必须由 Python 代码基于真实数据生成          |
| 图文幻觉 | 图表中文字变乱码、数值标注错误      | 用图像生成模型绘制含数据的图表  | 改用 Gemini Nano Banana（高保真文字渲染）     |
| 概念幻觉 | 混淆相似的临床术语或统计概念       | 解释机制、定义术语        | 用 NotebookLM 绑定文献来源，锚定回答至原文        |

## 使用 NotebookLM 从源头消灭文献幻觉

 **NotebookLM 的核心价值：**与 ChatGPT/Claude 的"通用知识库"不同，NotebookLM 只从你上传的文献中回答问题。如果资料里没有，它会直接说"这份资料中未提及"，而不是编造一个听起来合理的答案。每条回答后都有精确到原文段落的来源标注，可点击直接跳转核查。

科研文献管理工作流：

- 在 <https://notebooklm.google.com/> 创建一个新笔记本（如"心衰预测研究"）。
- 上传你的核心参考文献（支持 PDF、Word、网页 URL、Google Docs）。

3. 设定 AI 角色（点击右上角对话设置）：

"你是一位严格的医学统计审稿人。针对我的每个问题，先给出结论，再列出 2-3 个来自上传文献的支撑证据，最后指出该结论在现有资料中最薄弱的地方。所有回答必须有来源标注。"

4. 直接提问，所有回答都锚定在你的文献范围内，**不会自由发挥**。

## 四、附录：常用资源速查表

### A. 学习资源

| 资源名称            | 链接/位置                                                                 | 适合阶段 |
|-----------------|-----------------------------------------------------------------------|------|
| Python 官方教程（中文） | <a href="https://docs.python.org/zh-cn">docs.python.org/zh-cn</a>     | 入门   |
| Hugging Face 文档 | <a href="https://huggingface.co/docs">huggingface.co/docs</a>         | 进阶   |
| 动手学深度学习         | <a href="https://d2l.ai/zh">d2l.ai/zh</a>                             | 进阶   |
| PubMed 文献检索     | <a href="https://pubmed.ncbi.nlm.nih.gov">pubmed.ncbi.nlm.nih.gov</a> | 全程   |

### B. 常用命令速查

bash

```
# 查看当前 Python 版本
python --version

# 查看已安装包列表
pip list

# 查看某个包的版本
pip show transformers

# 在 Jupyter 中安装包（临时）
!pip install 包名

# 查看 GPU 状态（需有 NVIDIA 显卡）
nvidia-smi
```



## C. 文件结构推荐

```
your_project/
├── data/
│   ├── raw/           # 原始数据（只读，不要修改）
│   └── processed/     # 处理后数据
├── notebooks/         # Jupyter 探索性分析
├── src/               # 正式代码
│   ├── data_utils.py  # 数据处理函数
│   ├── model.py       # 模型定义
│   └── train.py        # 训练脚本
├── outputs/           # 模型输出、图表
├── requirements.txt    # 环境依赖
└── README.md          # 项目说明
```

本文档基于课题组实战经验整理，希望有所帮助。如有错误，欢迎指正与更新。

最后更新: 2026年 | 作者: 张沐寒